

21. Pointer recap, and arrays of pointers

Herman Kamper

This lecture is mostly a recap. Pointers and `malloc` are the two things that trip people up most in this course, and the best cure I know is to go back to the very beginning, to variables and boxes, and then walk forward in small steps until `malloc`. So the first part of this lecture recaps almost everything we have done: variables, bytes, pointers, `malloc` and arrays, all in one picture.

Then there is one new thing, arrays of pointers, which is easy if you followed the recap. And at the end there is a longer example (shuffling and dealing a deck of cards) which pulls the whole course up to now together.

If something in the first part does not make sense, stop there and sort it out before moving on.

Everything in one picture

Here is the program we are going to walk through. It does nothing useful. Every line puts one more thing in memory.

```
int x = 9014;
char y = 'a';
printf("%d", x);

int *foo;
foo = &x;
printf("%d", *foo);

double *foo2;
foo2 = malloc(sizeof(double));
*foo2 = 16.5;

int *foo3;
foo3 = malloc(2 * sizeof(int));
*foo3 = 7;           // foo3[0] = 7;
*(foo3 + 1) = 9;     // foo3[1] = 9;

free(foo2);
```

And below is what it does to memory. The rest of this section is a slow walk through this picture.

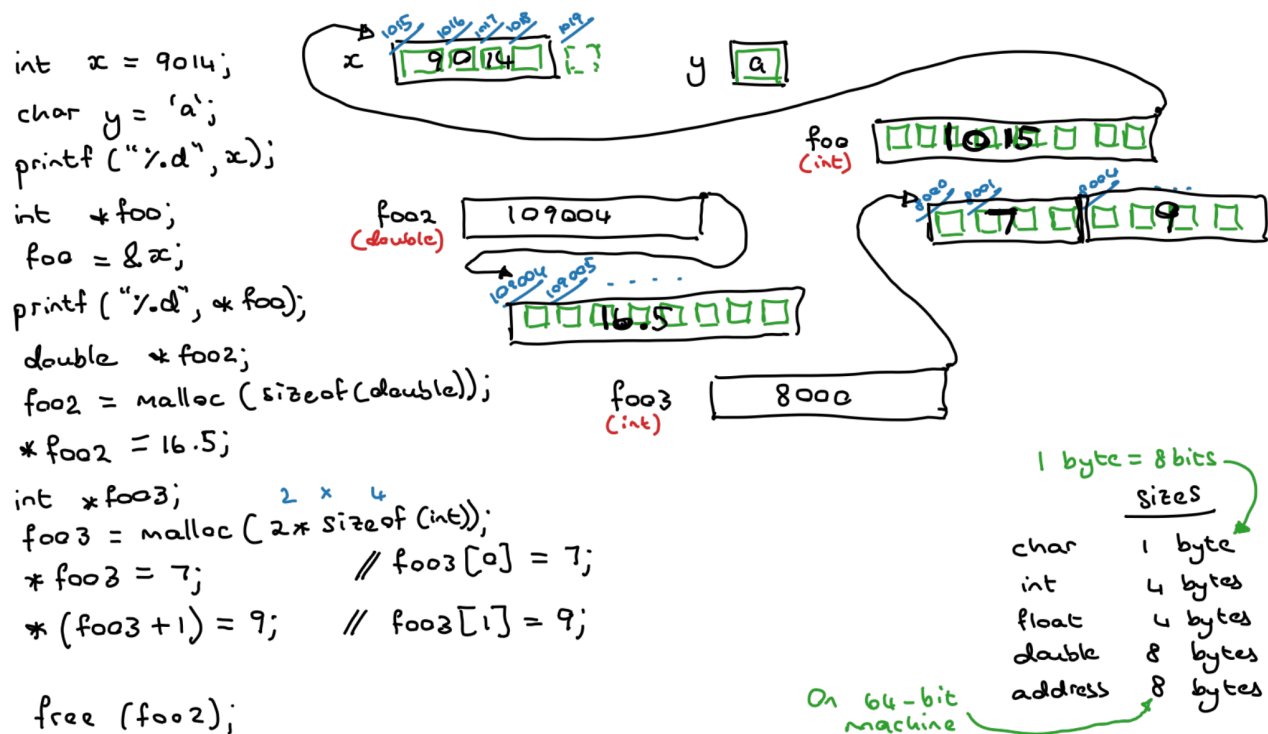


Figure 1: The whole program in memory. On the left is the code, in the middle the boxes it creates, and on the right the sizes of the things we have been using. Green is the bytes making up a box; blue is an address; red is the type that a pointer points to.

Boxes are made of bytes

Start with the first two lines. `int x = 9014;` creates a box in memory called `x` with 9014 inside it. The `char y = 'a';` creates a different box called `y` with `a` inside it. You have seen this a hundred times.

Except that I have been lying to you slightly. There is not really one box called `x`. There is a whole run of little boxes, each holding eight ones and zeros. Eight bits is what we call a byte, and one byte is the smallest thing in memory that has an address of its own. An `int` takes four bytes. So the `x` box is really four byte-sized boxes sitting next to each other, and each of those four has its own address: 1015, 1016, 1017 and 1018 in the picture. The value 9014 is written in ones and zeros across all four of them. The box right after, at 1019, is somebody else's problem (e.g. it could be used by C to store another variable later on).

A `char` takes one byte, so the `y` box really is just one box, sitting somewhere else in memory with an address of its own.

The sizes listed on the right of the picture are worth committing to memory. A `char` is 1 byte, an `int` is 4 bytes, a `float` is 4 bytes, a `double` is 8 bytes, and an address is 8 bytes on a 64-bit machine.

Now look at `printf("%d", x);` and ask how it actually manages to print 9014. It goes to where `x` starts, at 1015. It knows an `int` is four bytes, so it reads all four of them, takes those 32 ones

and zeros together, converts them to a whole number and prints that. The size of the type is what told it how far to read (in this case: four bytes, or four green boxes in the picture).

The pointer box

Next line: `int *foo;`. This creates a new box called `foo`, somewhere else in memory. At the moment there is nothing useful in it, just whatever was lying around in those bytes before.

How big is the `foo` box? Eight bytes, because what we are going to put in it is an address, and an address takes eight bytes on a 64-bit machine. The size of this box has nothing to do with the `int` at the front of the declaration. Any pointer box, regardless of what it points to, will take up eight bytes.

Then `foo = &x;` puts the address of `x` into the `foo` box. The address of `x` is 1015, so 1015 is what goes in, written in ones and zeros across the eight bytes of `foo`.

And then I draw a little arrow from `foo` to `x`. That arrow does not exist. There is no arrow anywhere in the machine. It is something I draw to remind myself that the number in this `foo` box is an address, and that an address is where some other box lives.

Two small things about that arrow while we are here. First, it does not really point at `x`. It points at the *first byte* of `x`, at 1015. You can think of the two interchangeably most of the time, but 1015 is what is actually being pointed to (because this is the actual value inside `foo`).

Second, and this is subtle: there is no address for `x` as a whole. Addresses belong to bytes. What you have is the address where `x` starts, plus the knowledge that `x` is an `int` and therefore four bytes long. You implicitly know every byte of `x` (because you know where it starts and how many bytes it spans), but there is no single number that means “all of `x`”.

How C knows how much to read

Now the second `printf`. `printf("%d", *foo);` prints 9014, the same as the first `printf` did. Work out carefully how it gets there. The `*` says: go to the box I am pointing at. Which box is that? The one at address 1015, the first byte of `x`. And then it reads from there and prints what it finds.

But how many bytes does it read? Here is the question that everything hinges on. Every pointer box is eight bytes, always, because every pointer box holds an address. A `char *` is eight bytes, an `int *` is eight bytes and a `double *` is eight bytes. The size of the pointer tells you nothing about what it is aimed at.

What tells C how much to read is the type in the declaration. We wrote `int *foo`, so `foo` is a pointer to an `int`. So when we dereference it, C reads an `int`-sized chunk (four bytes) starting at 1015. That is the point of the `int` in `int *foo`. It does not change the box, it does not change the address, it only says how big a chunk to take when you open the thing at the other end (via dereferencing).

malloc, once more

Now the lines that cause trouble. There are two mallocs in this program. Let's take them one at a time.

```
double *foo2;  
foo2 = malloc(sizeof(double));
```

The line declaring the pointer is nothing new: `foo2` is just another pointer box, eight bytes big, holding an address, aimed at a double.

The `malloc` line is the one to think about. Forget for a moment that a pointer is involved and look only at what is inside the parentheses. You are telling `malloc` how many bytes to block off. A `double` is eight bytes, so `sizeof(double)` is 8, and `malloc` goes and finds eight bytes somewhere in memory, marks them as yours, and promises that nothing else, not another program and not the rest of this one, will put anything in them. In the picture those eight bytes start at 109004, so they run 109004, 109005, and so on up to 109011.

What does `malloc` give you back? It returns the address of the first byte in the block it just allocated. That is all it returns: one number, 109004. Which is exactly why the result goes into a pointer box, because an address is what a pointer box holds. So 109004 lands in `foo2`, and now I can draw another arrow that does not exist.

Let me be clear. There are not two `foo2` boxes in the picture. There is one `foo2` box, the little one holding 109004. And then there is a block of memory somewhere else that it points at. Compare that with the left of the picture: `int x = 9014;` made a box and gave it a name, `x`. `malloc` made a block and gave it no name at all. The only way you will ever reach that block again is through the address sitting in `foo2`. Lose that address and the block is still reserved, still off-limits to everyone else, and completely unreachable. That is a memory leak, and this is why the program should end with

```
free(foo2);
```

`free` is how you tell the operating system that you are letting go, and that the memory can be handed to someone else. (The program in the picture frees `foo2` and stops there, which is a bug: the block we are about to allocate for `foo3` needs a `free(foo3);` of its own. Every `malloc` needs a matching `free`. Sorry that I forgot it.)

Putting something in the block

The next line is

```
*foo2 = 16.5;
```

By now you can work out exactly what this does. `foo2` holds 109004, and the `*` says: go to the box at that address. How big is that box? Eight bytes, because `foo2` is a `double *`. So 16.5 is written across all eight bytes, from 109004 up to 109011.

It is worth being precise about “across all eight”, because it is a natural thing to misunderstand. The value does not get packed into the first byte or two with the rest left empty. A `double` has

its own way of writing a number in 64 ones and zeros, and you need all eight of those bytes together to get 16.5 back out again.

A block of two

Now the second malloc:

```
int *foo3;  
foo3 = malloc(2 * sizeof(int));
```

An int is four bytes, so `2 * sizeof(int)` is 8, and once again malloc blocks off eight bytes. In the picture they start at 8000, so they run 8000 up to 8007. And 8000 is the number that comes back from malloc and goes into the foo3 box.

So far this is identical to foo2: an eight-byte pointer box holding the address of an eight-byte block. What differs is what we intend to do with those eight bytes. Last time they were one double. This time they are two ints. To understand this, look at the next line:

```
*foo3 = 7;
```

foo3 holds 8000. The * opens the box at 8000, and because foo3 is an int *, that box is four bytes wide. So 7, in binary, goes into bytes 8000 to 8003. You could equally have written that line as `foo3[0] = 7;`. Stop and make sure you see what happened here. The value 7 did not get written to the whole chunk reserved by malloc. Instead (following exactly the same logic as applied to foo2), the 7 only landed inside the first four bytes in the malloc-allocated block.

The second int line is also interesting:

```
*(foo3 + 1) = 9;
```

What is `foo3 + 1`? It is not 8001. foo3 points at ints, so adding one moves on by one int, which is four bytes, giving 8004. Then the * opens a four-byte box starting there, and 9 goes into bytes 8004 to 8007.

There is also another notation to do exactly the same thing:

```
foo3[1] = 9;
```

`pointer[i]` means exactly `*(pointer + i)`: take the address, add i blocks of whatever the pointer is aimed at, then go to that box. This is just how arrays in C work – because in C, arrays are pointers.

Notice the pointer's type doing its two jobs again. Put foo2 and foo3 side by side: both are eight-byte boxes, both hold an address, and both point at eight-byte blocks. The only difference between them is the type in the declaration. Dereferencing foo2 reads or writes eight bytes at a time and `foo2 + 1` would step eight bytes; dereferencing foo3 touches four bytes and `foo3 + 1` steps four. How far + 1 moves, and how much * reads or writes: that is what the type in the pointer declaration is for.

Nothing stops you

One warning to close this off. We allocated two ints for foo3. Suppose you write this:

```
*(foo3 + 2) = 101;
```

That steps on to 8008, which is one byte past the end of what you were given, and writes four bytes there. C will let you. It will not warn you, and sometimes it will not crash. One of three things happens: the write lands in memory nobody is using and appears to work; the write lands in memory this program is using for something else and quietly corrupts it; or it lands somewhere the operating system will not allow and you get a segmentation fault.

The second of those is the nasty one, because the program runs and the output looks right and the bug shows up much later somewhere else entirely. malloc gives you a block. Stay inside it. (C won't protect you from yourself.)

Arrays of pointers

Now the one new thing in this lecture, and it really is almost nothing. An array of pointers is an array whose elements happen to be pointers. It is all in the name.

```
/*  
 * Show an array of string pointers.  
 */  
  
#include <stdio.h>  
  
int main()  
{  
    char string1[] = "apples";  
    char string2[] = "pears";  
  
    char *listOfStr[2];  
  
    listOfStr[0] = string1;  
    listOfStr[1] = string2;  
  
    printf("%s\n", listOfStr[0]);  
  
    return 0;  
}
```

Build the new declaration up one piece at a time. If we had `char * listOfStr;` you would know exactly what is created: a box holding an address, aimed at a char. Now here we instead have `char *listOfStr[2];`. The [2] at the end says: make two of those, side by side, with the names `listOfStr[0]` and `listOfStr[1]`. That is the whole of the new syntax. Each element is eight bytes, because each element is a pointer (storing an address, taking eight bytes).

Now what does the rest of the program put in memory?

The first two lines create two character arrays: a, p, p, l, e, s and then the `\0` that marks the end of a string, and next to them p, e, a, r, s, `\0`. Each box in those arrays is one byte, because each holds a char.

Then `listOfStr[0] = string1;` does what it always does. `listOfStr[0]` is a pointer box, so an address goes in it, and `string1` on its own is the address of the first character of that array. If that array starts at 1014, then 1014 will go into the box. The next line does the same with `string2`, whose first character sits at 90112. All of it is drawn below.

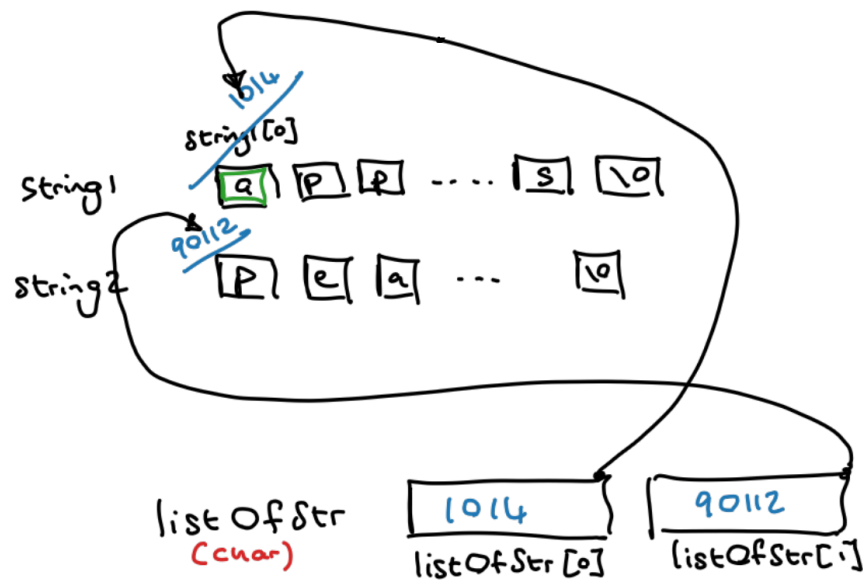


Figure 2: The two character arrays and the array of pointers. `listOfStr[0]` holds the address of the first character of `string1`, and `listOfStr[1]` holds the address of the first character of `string2`. The arrows, as always, do not exist.

And `printf("%s\n", listOfStr[0]);` prints

apples

because `%s` takes an address, walks forward from it printing characters, and stops when it hits the `\0`.

Look at how much is going on in those few lines. There is an array of pointers, and each of the things it points at is itself an array. But there is nothing in it you did not already know. The only new syntax was the `[2]`.

A last thing. The lines

```
char string1[] = "apples";  
char string2[] = "pears";
```

could also be written exactly equivalently as

```
char *string1 = "apples";  
char *string2 = "pears";
```

Go and think about it. Make sure you understand why.

Reading combined declarations

Here is the kind of line that appears in the card program at the end of this lecture:

```
const char *suitNames[4] = {"hearts", "diamonds", "clubs", "spades"};
```

I read these from the inside out, and honestly I often still have to write a small program to check that I have got it right. Work through it in pieces.

The part on the right, {"hearts", "diamonds", "clubs", "spades"}, creates four character arrays somewhere in memory, each ending in `\0`. We do not know or care where they are, but each of them has an address.

The part on the left declares the thing that will hold those addresses. Reading outward from the name: `suitNames` is the name; `[4]` says it is an array of four of them; the `*` says each element is a pointer; `char` says each points at characters; and `const` says we are promising not to change the characters being pointed at.

Put together: an array of four pointers to `const char`, with `suitNames[0]` holding the address of the first character of "hearts", `suitNames[1]` the address of "diamonds", and so on. It is exactly the picture from the previous section, with four strings instead of two.

If a line like this does not click immediately, that is normal. Give it an evening. Make sure after that evening you've got it.

Shuffling and dealing a deck of cards

Now a longer example that uses all of the above. The problem: build a 52-card deck, shuffle it and deal the cards out in order. This sounds intricate, and the trick, as usual, is to solve the problem on paper before writing any code. Most of the work is deciding how to represent the deck.

A card has a suit and a face: the ace of hearts, the queen of clubs, etc. There are four suits and thirteen faces, so let's represent the deck as a 4 by 13 array of `int`:

```
int deck[4][13] = {0};
```

Row 0 is hearts, row 1 diamonds, row 2 clubs, and row 3 spades. The columns run ace, deuce, 3, up to king. So `deck[0][0]` is the position for the ace of hearts and `deck[3][12]` is the position for the king of spades.

What goes *in* the boxes? Not the card, the card is already given by which box you are looking at. What goes in the box is where that card sits in the shuffled deck, a number from 1 to 52. A zero means we have not placed that card yet, which is why the array is initialised to all zeros.

So an ordered deck, with the ace of hearts first and the king of spades last, would look like this:

	A	2	3	4	5	6	7	8	9	10	J	Q	K
hearts	1	2	3	4	5	6	7	8	9	10	11	12	13
diamonds	14	15	16	17	18	19	20	21	22	23	24	25	26
clubs	27	28	29	30	31	32	33	34	35	36	37	38	39
spades	40	41	42	43	44	45	46	47	48	49	50	51	52

And shuffling is then nothing more than writing the numbers 1 to 52 into the same grid in a random order. One shuffle came out like this:

	A	2	3	4	5	6	7	8	9	10	J	Q	K
hearts	15	38	36	27	34	3	8	7	22	14	35	20	26
diamonds	40	47	42	19	29	16	12	2	39	4	30	52	13
clubs	33	43	21	25	9	48	51	44	50	32	28	1	6
spades	5	10	41	23	46	11	31	49	24	18	17	37	45

Reading that: the first card of the shuffled deck is wherever the 1 is, which is row 2, column 11, which is the queen of clubs. The second card is wherever the 2 is: row 1, column 7, so the 8 of diamonds.

That is the solution! To shuffle, walk through the card positions 1 to 52 and for each one pick a random empty slot in the grid and write the number there. To deal, walk through the positions 1 to 52 again and for each one search the grid for that number, then print the card whose row and column you found it at.

That is where the arrays of pointers come in. The grid holds numbers, but we want to print “queen of clubs”, so we need a way to turn a row into a suit name and a column into a face name:

```
const char *suitNames[4] = {"hearts", "diamonds", "clubs", "spades"};
const char *faceNames[13] = {"ace", "deuce", "3", "4", "5", "6", "7", "8",
                              "9", "10", "jack", "queen", "king"};
```

Now `suitNames[row]` is the suit and `faceNames[column]` is the face, and the whole program follows:

```

/*
 * Shuffles and deals cards.
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void shuffle(int deck[][13]);
void deal(int deck[][13], const char *suitNames[], const char *faceNames[]);

int main()
{
    const char *suitNames[4] = {"hearts", "diamonds", "clubs", "spades"};
    const char *faceNames[13] = {"ace", "deuce", "3", "4", "5", "6", "7", "8",
                                   "9", "10", "jack", "queen", "king"};

    int deck[4][13] = {0};

    srand(time(0));

    shuffle(deck);
    deal(deck, suitNames, faceNames);

    return 0;
}

void shuffle(int deck[][13])
{
    int card, suit, face;

    // For each of the 52 card positions
    for (card = 1; card <= 52; card++)
    {
        // Pick random slots until we find an empty one
        do
        {
            suit = rand() % 4;
            face = rand() % 13;
        }
        while (deck[suit][face] != 0);

        // Put this card position into that slot
        deck[suit][face] = card;
    }
}

```

```

}

void deal(int deck[][13], const char *suitNames[], const char *faceNames[])
{
    int card, row, column;

    // For each of the 52 card positions, in order
    for (card = 1; card <= 52; card++)
    {
        // Search the whole deck for it
        for (row = 0; row < 4; row++)
        {
            for (column = 0; column < 13; column++)
            {
                if (deck[row][column] == card)
                {
                    printf("%s of %s\n", faceNames[column], suitNames[row]);
                }
            }
        }
    }
}

```

The shuffle from the grid above deals like this:

```

queen of clubs
8 of diamonds
6 of hearts
10 of diamonds
ace of spades
king of clubs
...

```

A few things to notice. `srand(time(0))` seeds the random number generator from the clock, so you get a different shuffle each run; replace it with `srand(42)` while you are debugging and you will get the same shuffle every time. The `do ... while` in `shuffle` keeps picking random slots until it finds one still holding a zero, so no two cards land in the same slot. And `deal` is deliberately dumb: for each position it searches all 52 slots. That is 52 times 52 comparisons, which is not that much here, but it is the kind of thing worth noticing (there is much better ways to do this; think about it).

This is a good program to go and write yourself rather than just read.

Looking forward

That is the pointer material. None of it is complicated on its own: a box holds bytes, a pointer box holds an address, the type says how many bytes to read at the other end when we dereference, and malloc hands you the address of a block it has set aside for you. Everything else in this lecture was those four facts applied twice over.

Exercises

1. Write out the recap program from the first section and add the missing `free(foo3);`. Then print `foo`, `foo2`, `foo2 + 1`, `foo3`, `foo3 + 1` and `&x`, all with `%p`. Check that the gap between `foo2` and `foo2 + 1` is `sizeof(double)` and the gap between `foo3` and `foo3 + 1` is `sizeof(int)`.
2. Print `sizeof(foo)`, `sizeof(foo2)`, `sizeof(foo3)` and `sizeof(char *)`. Explain why all four are the same even though they are aimed at different things.
3. Change `foo3` to `double *foo3 = malloc(2 * sizeof(double));`, write 16.5 and 2.5 into the two slots and print them back. Print `foo3` and `foo3 + 1` and explain the gap.
4. Add `*(foo3 + 2) = 101;` to the program and run it several times. Describe what happens, and say why “it seems to work” is not the same as “it is correct”.
5. In the `listOfStr` program, print `listOfStr[1][2]` with `%c` and work out what it should be before you run it. Then print `listOfStr[0]` and `listOfStr[1]` with `%p` and explain what those two numbers are.
6. Extend the `listOfStr` program to five fruit, then print them all with a loop. Write the loop twice, once with `listOfStr[i]` and once with `*(listOfStr + i)`.
7. Type up the card program and run it. Then replace `srand(time(0))` with `srand(42)` and confirm that two runs now give the same deal.
8. The `deal` function searches the whole 4 by 13 grid for every one of the 52 positions. Rewrite it so that it goes through the grid only once, by first building a 53-element array that maps each position to its row and column. Compare the amount of work the two versions do.